

Efficient Implementation of ARIA on ARMv8 via Cryptographic Extensions

Myoungsu Shin¹, Hanbeom Shin², Insung Kim² and Dongjae Lee^{a,3}

¹ ENERZAi Inc., Republic of Korea

² Korea University, Republic of Korea

³ Kangwon University, Republic of Korea

Abstract. The ARIA block cipher is the Korean national standard (KS X 1213) and an IETF standard (RFC 5794). Despite its widespread use, research on efficient implementation for modern ARMv8 processors has remained limited compared to AES, which benefits from dedicated hardware instructions. The best prior ARMv8 result by Eum et al. reported 0.573 cycles per byte (cpb); however, through direct communication with the authors and independent re-evaluation, we confirmed that this published figure reflects a measurement error and that the actual cost is 5.845 cpb. In this paper, we present a 16-way parallel ARIA implementation on ARMv8 NEON that evaluates all four ARIA S-boxes through the **AESE/AESD** cryptographic extensions, eliminating S-box table lookups from the round function entirely. Two of the S-boxes reduce to a single **AESE/AESD** instruction, while the remaining two are computed via a nibble-split decomposition of the underlying multiplicative inverse, requiring only two NEON registers of precomputed constants per S-box. Combined with a byte-sliced data layout and a 64-instruction transposition butterfly, our implementation achieves 1.483 cpb for ARIA-128 on the Apple M1—a 3.94× speedup over the corrected prior result. Multi-threaded CTR-mode measurements demonstrate near-linear scalability, reaching 6.67 GB/s with 4 threads on the performance cores and 8.33 GB/s with 8 threads. On the ARM Cortex-A76 (Raspberry Pi 5), the implementation achieves 3.586 cpb and scales to 2.36 GB/s with 4 threads.

Keywords: ARIA · ARMv8 · NEON · cryptographic extensions · byte-slicing · AESE/AESD

1 Introduction

The ARIA block cipher [KKP⁺04] is the Korean national encryption standard (KS X 1213) and is formally specified as IETF RFC 5794 [Lee10]. Despite being a mandatory requirement for national public infrastructure and TLS deployments in Korea [KLPK11], research on its efficient implementation for modern processors has lagged significantly behind AES, which benefits from dedicated hardware support across nearly all performance-oriented architectures. The proliferation of ARMv8-based processors—from Apple’s M-series and AWS Graviton in cloud servers to Qualcomm Snapdragon in mobile devices and Broadcom BCM2712 (Cortex-A76) in embedded platforms—makes high-performance ARIA on ARM a practical necessity.

ARIA shares key structural properties with AES [Nat01]: both are substitution-permutation networks over $\text{GF}(2^8)$, and ARIA’s first S-box S_1 is mathematically identical to AES SUBBYTES [KKP⁺04]. However, its unique substitution layer and 16×16 binary

E-mail: houma757@gmail.com (Myoungsu Shin), dongjae.lee@kangwon.ac.kr (Dongjae Lee)

^aCorresponding author.

involution diffusion matrix have historically prevented ARIA from benefiting directly from the dedicated AES hardware instructions (AESE/AESD) available on ARMv8. Prior implementations [EKK⁺22] have relied on memory-resident S-box table lookups, incurring significant memory traffic and latency overhead.

In this paper, we address this gap by presenting a 16-block parallel ARIA implementation for ARMv8 that repurposes the AESE/AESD cryptographic extensions to accelerate all four ARIA S-boxes. By deriving a nibble-split affine decomposition for the non-AES S-boxes, we eliminate S-box memory traffic entirely and achieve performance levels competitive with hardware-accelerated AES.

Our main contributions are as follows:

1. **Hardware-accelerated byte-sliced ARIA on ARMv8.** We present the first ARIA implementation that uses ARMv8 AESE/AESD instructions for S-box computation within a 16-block byte-sliced framework. S_1/S_1^{-1} are evaluated in 1–2 instructions via hardware AES operations; S_2 and S_2^{-1} are computed in 7 and 6 instructions respectively, using a nibble-split decomposition of the combined affine map whose constants are held permanently in NEON registers, eliminating S-box memory traffic from the round function entirely.
2. **Reduction of the SR side-effect to a single TBL correction.** AESE/AESD fuse SUBBYTES with AES SHIFTRROWS (SR) as a side-effect that, in our byte-sliced context, permutes block-lanes within a register. We reduce this correction to a single SR^2 TBL per AESE-based S-box and absorb the residual offset into the round-key schedule, leaving subsequent KEYXOR and diffusion operations unaffected.
3. **Significant speedup and correction of prior results.** Our implementation achieves **1.483 cpb** for ARIA-128 on the Apple M1, a **3.94×** improvement over the best prior ARMv8 result. In multi-threaded CTR mode, the implementation scales near-linearly to **6.67 GB/s** with 4 threads on the Firestorm performance cores (3.79× vs. single-thread) and reaches **8.33 GB/s** with 8 threads on the full M1 cluster. Through communication with the original authors and independent benchmarking, we confirm that the actual performance of Eum et al.’s [EKK⁺22] implementation is 5.845 cpb rather than the published 0.573 cpb.
4. **Embedded ARMv8 benchmarks without FEAT_SHA3.** We also benchmark our implementation on the ARM Cortex-A76 (Raspberry Pi 5), which lacks the EOR3 instruction; each three-operand EOR3 in the diffusion layer is replaced by two two-operand EORs. On this platform we achieve **3.586 cpb** (5.82× faster than optimized reference C) and scale to **2.36 GB/s** with 4 threads.

2 Related Work

Block cipher optimization on ARMv8. The ARMv8 architecture introduced dedicated cryptographic extensions (AESE/AESD, PMULL, SHA-2/3) on top of a wider 128-bit ASIMD register file, both of which have been exploited for block-cipher acceleration over the past decade. AES itself, the natural target of these instructions, has been accelerated on ARMv8 through hand-tuned routines in production cryptographic libraries such as OpenSSL and BoringSSL, providing the de facto reference point against which other block ciphers on this platform are compared. Gouvêa and López [GL15] demonstrated the practical impact of these extensions early on by implementing GCM over AES-128 at 1.71 cpb—about 9× faster than the ARMv7 baseline—using PMULL for binary-field multiplication and AESE for the underlying cipher. For ciphers without a dedicated instruction path, NEON’s data-parallel pipeline has been the primary acceleration vehicle. Song and Seo [SS20] applied task- and data-parallel ASIMD processing to the ARX-based block ciphers HIGHT and CHAM,

reporting up to a $38\times$ speedup over prior reference code while integrating fault-attack countermeasures based on intra-instruction redundancy. Kwon et al. [KKE⁺22] optimized SM4 across AVR, RISC-V, and 64-bit ARM, reaching 8.62 cpb on ARM through parallel NEON computation. Building on the same platform, Guo [Guo22] further improved constant-time SM4 on ARM NEON to 6.74 cpb by combining an algebraic simplification of the SM4 S-box with novel L1-cache exploitation techniques. For lightweight ciphers, Lee et al. [LSS⁺25] examined how the interleave size of double-block parallel encryption interacts with the bit-permutation layer of GIFT-128 on 64-bit ARM, identifying six possible interleave configurations and reporting a 12% gap between the best and worst, with the best configuration running roughly twice as fast as single-block encryption—a concrete demonstration that data-layout choice can be a first-order performance lever even for ciphers without a dedicated SIMD-friendly substitution layer. Specifically for ARIA, Eum et al. [EKK⁺22] presented the first 16-block parallel ARMv8 implementation, evaluating all four S-boxes through NEON TBL lookups on memory-resident tables. Although their byte-sliced layout processes all blocks simultaneously, the full S-box tables must be reloaded from memory on every round, incurring 192 `ld1` instructions per 16-block batch. Their reported 0.573 cpb on the Apple M1 was independently re-evaluated at 5.845 cpb through our benchmarking (Section 5.1). None of these ARMv8 efforts exploit the AESE/AESD instructions to evaluate ARIA’s substitution layer, which is the gap this paper addresses.

ARIA optimization across computing environments. Beyond ARMv8, ARIA has been optimized across x86 servers, low-end microcontrollers, RISC-V processors, and GPU back-ends. Among them, Yoo et al. [YKC23] is the most directly relevant to our approach: they accelerated ARIA in the Linux kernel crypto subsystem with AVX, AVX2, and AVX-512 parallel versions, providing two ARIA-specific substitution paths leveraging AES-NI and GFNI together with an affine-transformation optimization on the AES-NI path, reporting up to a $10.6\times$ speedup over the generic implementation. The affine relation between S_1 and S_2 that underlies their AES-NI construction provides the algebraic foundation we adapt to ARMv8; however, since the GFNI instruction has no ARMv8 equivalent, their implementation cannot be ported directly, motivating the nibble-split affine decomposition developed in Section 4.2. On embedded and RISC-V targets, prior work has focused on memory and instruction scheduling [SKJ⁺21] or on extending the ISA itself [KKS21, YIK24], while GPU implementations [ESKS25, ESSS25] have explored shared-memory and kernel-level optimizations to maximize aggregate throughput on bandwidth-bound parallel hardware—a regime that does not address the per-core CPU efficiency that is the focus of this paper.

3 Preliminaries

3.1 The ARIA Block Cipher

ARIA [KKP⁺04, Kor04] is a Korean national standard block cipher (KS X 1213-1) based on a substitution–permutation network (SPN). It operates on 128-bit blocks with key lengths $k \in \{128, 192, 256\}$ bits, performing $N_r \in \{12, 14, 16\}$ rounds respectively. Figure 1 gives a structural overview.

Each of the first $N_r - 1$ rounds applies three operations in sequence:

1. **Key addition** (KEYXOR): the 128-bit round key rk_r is XORed with the current 16-byte state.
2. **Substitution layer** (SUBLAYER): each of the 16 state bytes is independently transformed by one of four S-boxes in a fixed pattern. Odd-numbered rounds use pattern SL1; even-numbered rounds use SL2.

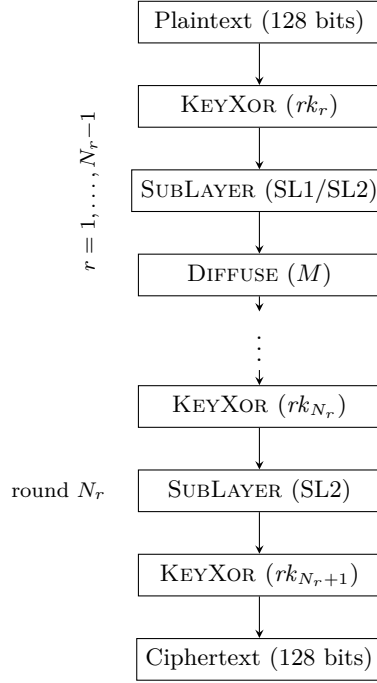


Figure 1: ARIA round structure for $N_r \in \{12, 14, 16\}$ rounds.

3. **Diffusion layer** (DIFFUSE): a fixed 16×16 binary involution matrix M over $\text{GF}(2)$ with Hamming weight 7 in every row.

The final round ($r = N_r$) omits DIFFUSE and appends a second KEYXOR with rk_{N_r+1} instead.

ARIA defines two primary S-boxes over the single field $\text{GF}(2^8)/p_1$ (the AES Rijndael field) and their functional inverses [KKP⁺04]:

$$S_1(x) = M_{\text{AES}} \cdot x_{p_1}^{-1} \oplus 0\text{x63} \quad (\text{AES SubBytes}), \quad (1)$$

$$S_2(x) = B \cdot x_{p_1}^{247} \oplus b, \quad (2)$$

$$S_1^{-1}(x) = (M_{\text{AES}}^{-1}(x \oplus 0\text{x63}))_{p_1}^{-1} \quad (\text{AES InvSubBytes}), \quad (3)$$

$$S_2^{-1}(x) = (B^{-1}(x \oplus b))_{p_1}^{223}, \quad (\text{functional inverse of } S_2) \quad (4)$$

where $p_1(x) = x^8 + x^4 + x^3 + x + 1$ is the AES irreducible polynomial, M_{AES} is the 8×8 AES affine matrix, and the ARIA-specific 8×8 matrix B and constant b are given in [KKP⁺04]. The exponents satisfy $x_{p_1}^{247} = (x_{p_1}^{-1})^8$ and $x_{p_1}^{223} = (x_{p_1}^{-1})^{32}$, since $|\text{GF}(2^8)^*| = 255$ and $-8 \equiv 247$, $-32 \equiv 223 \pmod{255}$. Equation (4) follows from inverting (2) and using $247 \cdot 223 \equiv 1 \pmod{255}$. Because S_1 shares both M_{AES} and the constant 0x63 with AES SUBBYTES, the two are byte-wise identical; analogously $S_1^{-1} = \text{INVSUBBYTES}$. We therefore use the ARIA notation S_1/S_1^{-1} throughout the paper and retain SUBBYTES/INVSUBBYTES only when directly quoting the ARMv8 specification.

All four S-boxes share the multiplicative inverse $x_{p_1}^{-1}$ in $\text{GF}(2^8)/p_1$ as their single nonlinear primitive. S_1 applies it directly; S_2 wraps it with the Frobenius automorphism $\sigma_3 : y \mapsto y^8$ of $\text{GF}(2^8)/p_1$ via $x^{247} = \sigma_3(x^{-1})$. The inverse S-boxes S_1^{-1} and S_2^{-1} apply the inverse function to an affine transformation of their input (with S_2^{-1} also wrapping the inverse with $\sigma_5 : y \mapsto y^{32}$, the inverse Frobenius automorphism of σ_3 in the Galois group). This unification is the key insight enabling hardware acceleration: any processor

	SL1 (odd rounds)				SL2 (even rounds)			
	col $j=0$	col $j=1$	col $j=2$	col $j=3$	col $j=0$	col $j=1$	col $j=2$	col $j=3$
row $k=0$	S_1 b_0	S_2 b_1	S_1^{-1} b_2	S_2^{-1} b_3	S_1^{-1} b_0	S_2^{-1} b_1	S_1 b_2	S_2 b_3
row $k=1$	S_1 b_4	S_2 b_5	S_1^{-1} b_6	S_2^{-1} b_7	S_1^{-1} b_4	S_2^{-1} b_5	S_1 b_6	S_2 b_7
row $k=2$	S_1 b_8	S_2 b_9	S_1^{-1} b_{10}	S_2^{-1} b_{11}	S_1^{-1} b_8	S_2^{-1} b_9	S_1 b_{10}	S_2 b_{11}
row $k=3$	S_1 b_{12}	S_2 b_{13}	S_1^{-1} b_{14}	S_2^{-1} b_{15}	S_1^{-1} b_{12}	S_2^{-1} b_{13}	S_1 b_{14}	S_2 b_{15}

Figure 2: SubLayer S-box assignment within one 16-byte block. Byte b_{4k+j} sits at row k , column j . Blue: F_1 -based (S_1/S_1^{-1}); red: F_2 -based (S_2/S_2^{-1}).

instruction that computes the AES-field inverse can, in principle, serve all four ARIA S-boxes once the appropriate affine and Frobenius matrices are absorbed into surrounding operations.

Within a single 128-bit block, the 16 state bytes are arranged as a 4×4 grid: byte b_{4k+j} sits at row k , column j ($k, j \in \{0, 1, 2, 3\}$). The applied S-box depends only on the column index j , so all four bytes sharing a column use the same S-box. Figure 2 shows the assignment pattern for both SL1 (odd rounds) and SL2 (even rounds); SL2 is SL1 with each S-box replaced by its functional inverse. The F_1 -based S-boxes (S_1/S_1^{-1}) are accelerated directly by hardware AESE/AESD, while the F_2 -based S-boxes (S_2/S_2^{-1}) are computed via our nibble-split affine decomposition (Section 4.2).

The diffusion layer computes $\mathbf{y} = M \cdot \mathbf{x}$, where M is a fixed 16×16 binary matrix over $\text{GF}(2)$ shown in Equation (6) [KKP⁺04]. M is an involution ($M^2 = I_{16}$, i.e., $M = M^{-1}$), so the same operation serves both encryption and decryption. Each row and column of M has Hamming weight 7, meaning each output byte is the XOR of exactly 7 input bytes. A naïve implementation requires $16 \times 6 = 96$ XOR operations.

Inspection of M reveals a four-fold symmetry: the 16 rows partition into four groups of 4, where each group shares a common 4-input XOR (t -value). The four t -values are:

$$\begin{aligned} t_0 &= x_3 \oplus x_4 \oplus x_9 \oplus x_{14}, & t_1 &= x_2 \oplus x_5 \oplus x_8 \oplus x_{15}, \\ t_2 &= x_1 \oplus x_6 \oplus x_{11} \oplus x_{12}, & t_3 &= x_0 \oplus x_7 \oplus x_{10} \oplus x_{13}. \end{aligned} \quad (5)$$

With these intermediates, every output reduces to $y_i = t_a \oplus x_j \oplus x_k \oplus x_l$ for appropriate a, j, k, l . For example, $y_0 = t_0 \oplus x_6 \oplus x_8 \oplus x_{13}$ and $y_3 = t_3 \oplus x_5 \oplus x_{11} \oplus x_{14}$. This reduces the total cost to $4 \times 3 + 16 \times 3 = 60$ scalar XOR operations [KKP⁺04, EKK⁺22], and is the starting point for our SIMD diffusion (Section 4.3).

The matrix M is given explicitly by [KKP⁺04]:

$$M = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}. \quad (6)$$

Table 1: Key ARMv8 instructions with Apple M1 Firestorm timings [Joh21].

Instruction	Operation	Latency	Throughput
AESE	XOR $\rightarrow$ SubBytes $\rightarrow$ SR	2	1
AESD	XOR $\rightarrow$ InvSubBytes $\rightarrow$ SR ⁻¹	2	1
TBL (1-reg)	16-entry byte lookup	3	1
TRN1/TRN2	Interleave even/odd elements	1	2
EOR3	Three-operand XOR	1	1
EOR	Two-operand XOR	1	3

Latency in cycles; throughput in instructions per cycle.

The four t -values correspond to a row grouping of M : rows $\{0, 5, 11, 14\}$ share t_0 , rows $\{1, 4, 10, 15\}$ share t_1 , rows $\{2, 7, 9, 12\}$ share t_2 , and rows $\{3, 6, 8, 13\}$ share t_3 .

3.2 ARMv8 Architecture and Cryptographic Extensions

The ARMv8 NEON/ASIMD architecture provides 32 vector registers v_0 – v_{31} , each 128 bits wide (16 bytes). Our ARIA implementation relies on four categories of NEON/Crypto instructions [Arm24], summarized in Table 1 with Apple M1 Firestorm-core timings [Joh21].

S-box evaluation via Cryptographic Extensions. The AESE and AESD instructions fuse a 128-bit XOR with SUBBYTES/INVSUBBYTES and SR/SR⁻¹ in a single instruction [Arm24]:

$$\begin{aligned} \text{AESE}(V_d, V_n) &= \text{SR}(\text{SUBBYTES}(V_d \oplus V_n)), \\ \text{AESD}(V_d, V_n) &= \text{SR}^{-1}(\text{INVSUBBYTES}(V_d \oplus V_n)). \end{aligned}$$

(These are restated as Eqs. (10)–(11) when used in derivations.) Because $S_1 = \text{SUBBYTES}$ (Section 3.1), these instructions directly compute the ARIA S_1/S_1^{-1} substitution on each of the 16 bytes in hardware. The unwanted SR side-effect and the exploitation of AESE/AESD for the remaining S-boxes are addressed in Section 4.2.

Affine decomposition via Table Lookup. The TBL instruction performs a parallel byte-wise lookup into a register-resident table of up to 64 bytes (1–4 consecutive registers). We use single-register TBL (16-entry table) for the nibble-split affine decomposition that maps S_1 output to S_2 (Section 4.2), as well as for undoing the SR permutation of AESE/AESD.

Data transposition. Converting between the standard block layout and the byte-sliced representation requires a 16×16 byte matrix transposition. The TRN1/TRN2 instructions interleave even- and odd-indexed elements from two source registers at configurable granularity (.16b, .8h, .4s, .2d), providing the building blocks for our butterfly transposition network (Section 4.1).

Diffusion via three-operand XOR. Since each output byte of the ARIA diffusion matrix is the XOR of 7 input bytes, efficient multi-operand XOR is critical. The EOR3 instruction (FEAT_SHA3) computes $V_d \leftarrow V_n \oplus V_m \oplus V_a$ in a single cycle, substantially reducing the instruction count for multi-operand XOR chains (Section 4.3).

4 Implementation

4.1 Byte-Sliced Data Layout

Our implementation adopts a byte-sliced representation [KS09, BS12] that processes 16 independent 128-bit blocks simultaneously. In this layout, NEON register v_j (for

$0 \leq j \leq 15$) holds byte position j from all 16 blocks:

$$v_j = (B_0[j], B_1[j], \dots, B_{15}[j]), \quad (7)$$

where $B_i[j]$ denotes the j -th byte of the i -th block. Each register thus contains 16 bytes, one from each block, all at the same byte position.

Each S-box application to byte position j across all 16 blocks requires only a single NEON instruction on v_j , amortizing overhead by $16\times$. Round keys are broadcast to byte-sliced form (16 copies per byte position) and precomputed.

Transpose16: matrix transposition. Incoming plaintext arrives in *block-major* form—each register v_i holds all 16 bytes of block B_i —and must be converted to the byte-sliced form of Equation (7), where v_j holds byte position j from all 16 blocks. This is a 16×16 byte matrix transposition, which we implement as a 4-stage butterfly network using the ARMv8 TRN1/TRN2 instructions at successively wider element granularities:

Stage 1 (.16b): Byte-pair interleave. For each pair (v_{2k}, v_{2k+1}) , TRN1.16b and TRN2.16b interleave even and odd bytes respectively.

Stage 2 (.8h): Halfword interleave. Pairs from Stage 1 are interleaved at halfword granularity.

Stage 3 (.4s): Word interleave. Results from Stage 2 are interleaved at word granularity.

Stage 4 (.2d): Doubleword interleave. Final stage produces the fully transposed output in v_0 – v_{15} .

Each stage consists of 16 TRN instructions (8 pairs of TRN1/TRN2), yielding a total of $4 \times 16 = 64$ instructions. Figure 3 illustrates the first two stages on a simplified 4×4 example (4 blocks of 4 bytes each); the full 16×16 transposition extends the same pattern across all four stages. A symmetric reverse transposition after the final round converts the byte-sliced ciphertext back to block-major form for output.

Unlike the classical butterfly (which requires all 16 upper registers as scratch, overwriting our S-box constants in v_{16} – v_{22}), our fused-stages variant restructures the computation into two passes. Pass A fuses Stages 1 and 2: each group of 4 input registers ($v_{4k} \dots v_{4k+3}$) is processed independently using 4 scratch temporaries (drawn from v_{24} – v_{27}). Pass B fuses Stages 3 and 4: each column of 4 registers is processed using 4 concurrently live temporaries; across the four columns the register allocator distributes them over $v_{24}, v_{25}, v_{28}, v_{29}$ to keep dependent groups independent. The fused variant therefore touches at most six scratch register names (v_{24} – v_{29} , with at most 4 live at any one time), preserving v_{16} – v_{23} throughout. The total instruction count remains 64 TRN instructions, and the critical path is unchanged (4 serial TRN latencies), since the M1’s out-of-order engine schedules independent groups in parallel. Listing 1 (Appendix B) shows one group from each pass.

4.2 S-Box Layer

All four ARIA S-boxes can be evaluated using a single hardware primitive—the ARMv8 AESE instruction—with zero S-box memory accesses. The derivation proceeds in four steps: (i) we show that all four S-boxes are affine maps of the same $\text{GF}(2^8)$ inverse; (ii) we show that AESE/AESD compute that inverse with a correctable side-effect, handling S_1 and S_1^{-1} in 2 and 1 instructions respectively; (iii) we derive S_2/S_2^{-1} as combined affine maps of the AESE output; (iv) we realize those maps via nibble decomposition in 7 instructions for S_2 and 6 for S_2^{-1} , keeping all constants register-resident.

Unified framework: all S-boxes via $x_{p_1}^{-1}$ and Frobenius. All four ARIA S-boxes are defined over the single field $\text{GF}(2^8)/p_1$ ($p_1 = x^8 + x^4 + x^3 + x + 1$, the AES Rijndael polynomial; cf. Section 3.1). The multiplicative inverse $x_{p_1}^{-1}$ in this field is the single

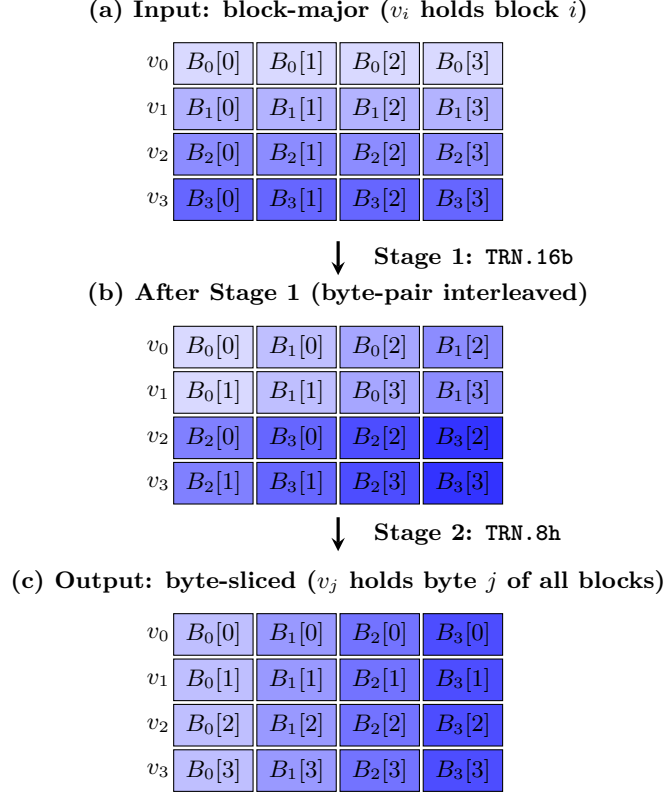


Figure 3: Byte-matrix transposition on 4 blocks of 4 bytes. $B_i[j]$ = byte j of block i (Eq. (7)).

nonlinear primitive underlying all four: S_1 and S_1^{-1} apply it directly, while S_2 and S_2^{-1} wrap it with the $\mathbb{F}_2$ -linear Frobenius automorphism $\sigma_3 : y \mapsto y^8$ of $\text{GF}(2^8)/p_1$ (or its inverse $\sigma_5 = \sigma_3^{-1} : y \mapsto y^{32}$ for S_2^{-1}), as derived below.

By the ARIA specification [KKP⁺04], $S_2(x) = B \cdot x_{p_1}^{247} \oplus b$. Since $|\text{GF}(2^8)^*| = 255$ and $-8 \equiv 247 \pmod{255}$, we have $x_{p_1}^{247} = (x_{p_1}^{-1})^8 = \sigma_3(x_{p_1}^{-1})$. Let C be the 8×8 binary matrix representing σ_3 on bit-vector representations. Then:

$$S_2(x) = B \cdot x_{p_1}^{247} \oplus b = B \cdot \sigma_3(x_{p_1}^{-1}) \oplus b = B \cdot C \cdot x_{p_1}^{-1} \oplus b = D \cdot x_{p_1}^{-1} \oplus b, \quad (8)$$

where $D = B \cdot C$ is a fixed 8×8 binary matrix that composes the ARIA affine transform B with the Frobenius matrix C . For S_2^{-1} , the symmetric situation arises from Eq. (4):

$$S_2^{-1}(y) = (B^{-1}(y \oplus b))_{p_1}^{223} = \sigma_5(B^{-1}(y \oplus b))_{p_1}^{-1}, \quad (9)$$

since $223 \equiv -32 \pmod{255}$ and $\sigma_5(z) = z^{32}$ is the inverse Frobenius automorphism of σ_3 . Equations (8)–(9) unify all four S-boxes under one nonlinear primitive: $x_{p_1}^{-1}$, the multiplicative inverse in the AES field, possibly composed with σ_3 (for S_2) or σ_5 (for S_2^{-1}).

AESE and AESD serve as hardware-level evaluators of $x_{p_1}^{-1}$, wrapped in a known affine layer and a byte permutation that can be corrected at negligible cost. The ARMv8 Architecture Reference Manual [Arm24] specifies:

$$\text{AESE}(V_d, V_n) = \text{SR}(\text{SUBBYTES}(V_d \oplus V_n)), \quad (10)$$

$$\text{AESD}(V_d, V_n) = \text{SR}^{-1}(\text{INVSUBBYTES}(V_d \oplus V_n)). \quad (11)$$

Setting $V_n = 0$ and applying $S_1 = \text{SUBBYTES}$, $S_1^{-1} = \text{INVSUBBYTES}$ (Section 3.1):

$$\text{AESE}(v, 0) = \text{SR}(S_1(v)), \quad (12)$$

$$\text{AESD}(v, 0) = \text{SR}^{-1}(S_1^{-1}(v)). \quad (13)$$

Inverting the affine wrapper of S_1 in (12) via the definition of S_1 in (1) recovers $x_{p_1}^{-1}$ explicitly:

$$x_{p_1}^{-1} = M_{\text{AES}}^{-1}(\text{SR}^{-1}(\text{AESE}(x, 0)) \oplus c_{\text{AES}}). \quad (14)$$

The SR side-effect must be undone to restore the canonical ARIA lane order. In standard block-major AES, SR permutes the 16 bytes of a single state. In our byte-sliced context, however, register v_j holds byte position j from 16 different blocks, so the same permutation instead reorders the 16 *block-lanes*:

$$(B_0[j], B_1[j], \dots, B_{15}[j]) \xrightarrow{\text{SR}} (B_{\pi(0)}[j], B_{\pi(1)}[j], \dots, B_{\pi(15)}[j]), \quad (15)$$

where π is the fixed permutation induced by SR on byte indices 0–15 (Figure 4).

The key observation is that SR has order 4, i.e., $\text{SR}^4 = I$ and hence $\text{SR}^{-1} = \text{SR}^3$. Since AESE leaves its outputs in SR state while AESD leaves its outputs in SR^{-1} state, the two groups of registers are offset from each other by SR^2 . Applying SR^2 as a single TBL to the AESE group yields $\text{SR}^2 \circ \text{SR} = \text{SR}^3 = \text{SR}^{-1}$, aligning it with the AESD group so that all 16 registers share a common lane state (Figure 4). The common SR^{-1} offset is harmless because it commutes with every operation in the round. Call an operation f on byte-sliced registers *position-independent* if its output at lane i depends only on the operand value(s) at lane i , not on the index i itself: S-box lookups (hardware AESE/AESD and the nibble-split TBL sequence) are unary position-independent, and XOR is binary position-independent between operands sharing the same SR offset. Every such f satisfies $f \circ \text{SR} = \text{SR} \circ f$, so post-multiplying the state by any power of SR is equivalent to pre-permuting the inputs of every subsequent operation by the same power.

Within a single round, the SR^2 TBL correction therefore brings the AESE and AESD groups into the common SR^{-1} lane state so that the diffusion XOR tree treats both groups identically.

Across rounds, this state cycles through the order-4 subgroup $\{I, \text{SR}, \text{SR}^2, \text{SR}^{-1}\}$, as each round adds a further AESE/AESD side-effect. We absorb this cyclic offset by pre-permuting each round key with the appropriate power of SR^{-1} at key-schedule time (Section 4.1), so that the KEYXOR of every round sees the state in its canonical position; the same pre-permutation also absorbs the implicit SR inherent to the byte-sliced layout.

We therefore precompute the correction vector k_{SR^2} and undo the permutation with a single TBL instruction:

$$\text{SR}^3 \circ S_1(v_j) = \text{TBL}(\text{AESE}(v_j, 0), k_{\text{SR}^2}), \quad (16)$$

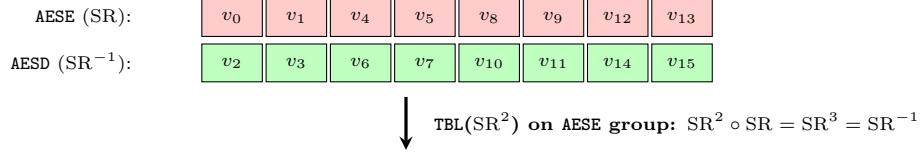
$$\text{SR}^3 \circ S_1^{-1}(v_j) = \text{AESD}(v_j, 0), \quad (17)$$

where we write $\text{SR}^3 = \text{SR}^{-1}$ so both right-hand sides leave the data in the common SR^{-1} lane state. Computing $\text{SR}^3 \circ S_1$ requires 2 instructions (AESE and an SR^2 TBL correction), while $\text{SR}^3 \circ S_1^{-1}$ requires a single AESD instruction because AESD already produces $\text{SR}^{-1}(S_1^{-1}(v_j))$; the residual SR^{-1} shared by both equations is absorbed into the key-schedule pre-permutation. Neither path incurs any S-box memory access.

S_2 : combined affine after AESE. With S_1 and S_1^{-1} handled, we turn to S_2 and S_2^{-1} . Equations (8) and (14) together give S_2 as an affine function of $S_1(x)$: substituting $x_{p_1}^{-1} = M_{\text{AES}}^{-1}(S_1(x) \oplus c_{\text{AES}})$ (rearranged from (1)) into (8) yields

$$S_2(x) = \bar{A}_{S_2} \cdot S_1(x) \oplus c', \quad \bar{A}_{S_2} = D \cdot M_{\text{AES}}^{-1} = B \cdot C \cdot M_{\text{AES}}^{-1}, \quad (18)$$

(a) After AESE/AESD: AESE group in SR, AESD group in SR^{-1}



(b) After SR^2 correction: all 16 registers in SR^{-1}

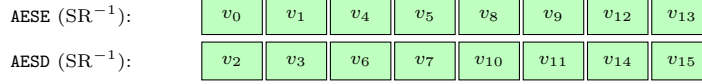


Figure 4: Lane-state alignment across the 16 data registers. **AESE** leaves its 8 outputs in SR state; **AESD** leaves its 8 outputs in SR^{-1} state. Applying SR^2 to the **AESE** group yields $\text{SR}^3 = \text{SR}^{-1}$, matching the **AESD** group and leaving all 16 registers in a common lane state for the subsequent diffusion.

where $c' = \bar{A}_{S_2} \cdot c_{\text{AES}} \oplus b$ is a fixed constant. Operationally, since **AESE** produces $\text{SR}(S_1(x))$ and $\bar{A}_{S_2}$ commutes with SR (a per-byte map commutes with any byte-position permutation), the implementation chain for S_2 is:

$$\text{AESE}(x, 0) \xrightarrow{\text{SR}^2 \text{ TBL}} \text{SR}^{-1}(S_1(x)) \xrightarrow{\bar{A}_{S_2}(\cdot) \oplus c'} \text{SR}^{-1}(S_2(x)),$$

mirroring the SR^{-1} lane state of Eq. (16)–(17); the residual SR^{-1} is absorbed into the key-schedule pre-permutation. The explicit form of $\bar{A}_{S_2}$ along with the constituent matrices B , C , M_{AES}^{-1} and the bit-vector convention is deferred to Appendix A; the resulting constant is $c' = 0\text{x}88$ (with $c_{\text{AES}} = 0\text{x}63$ and $b = 0\text{xe}2$).

S_2^{-1} : combined affine before AESD. For S_2^{-1} the situation is structurally different: because the inverse function appears *outside* the Frobenius wrapper in Eq. (9), the combined affine must be applied to the input *before* AESD rather than to its output. We seek $\bar{A}_{S_2^{-1}}$ and c'' such that

$$S_2^{-1}(y) = S_1^{-1}(\bar{A}_{S_2^{-1}} \cdot y \oplus c''). \quad (19)$$

Equating Eq. (9) with $S_1^{-1}(z) = (M_{\text{AES}}^{-1}(z \oplus c_{\text{AES}}))_{p_1}^{-1}$ and cancelling the outer $(\cdot)^{-1}$ on both sides:

$$\sigma_5(B^{-1}(y \oplus b)) = M_{\text{AES}}^{-1}(\bar{A}_{S_2^{-1}} y \oplus c'' \oplus c_{\text{AES}}).$$

Letting C^{-1} denote the matrix of $\sigma_5 = \sigma_3^{-1}$ and matching linear and constant parts gives

$$\bar{A}_{S_2^{-1}} = M_{\text{AES}} \cdot C^{-1} \cdot B^{-1}, \quad c'' = \bar{A}_{S_2^{-1}} \cdot b \oplus c_{\text{AES}}. \quad (20)$$

As before, the explicit form of $\bar{A}_{S_2^{-1}}$ is deferred to Appendix A; here $c'' = 0\text{x}04$. Since the AddRoundKey of **AESD** can absorb the constant c'' into the round-key schedule, the complete derivation chain for S_2^{-1} is:

$$y \xrightarrow{\bar{A}_{S_2^{-1}}(\cdot)} \bar{A}_{S_2^{-1}} y \xrightarrow{\text{AESD}(\cdot, c'')} \text{SR}^{-1}(S_2^{-1}(y)),$$

followed by the same lane-state alignment used for S_1^{-1} . This explains the encryption/decryption asymmetry of our implementation: S_2 applies its nibble affine *after* **AESE**, while S_2^{-1} applies it *before* **AESD**.

S_2/S_2^{-1} : nibble decomposition of $\bar{A}_{S_2}$. On ARMv8, where no GFNI equivalent exists, we realize the combined affine $\bar{A}_{S_2}$ via a nibble decomposition of the 8×8 GF(2)-linear map.

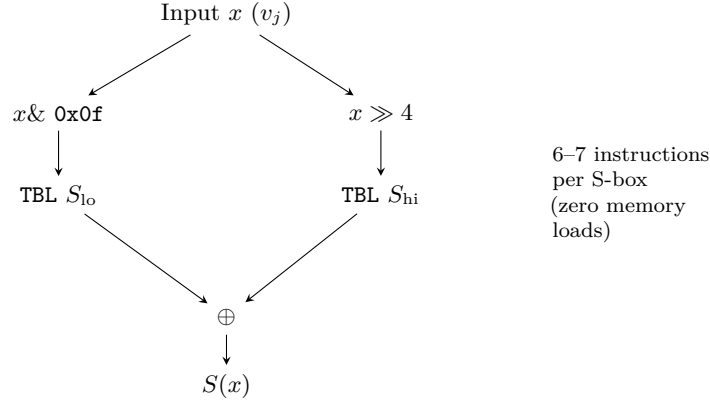


Figure 5: Nibble-split S-box computation for S_2 and S_2^{-1} .

Write any $x \in \{0, \dots, 255\}$ as

$$x = x_{lo} \oplus (x_{hi} \ll 4), \quad x_{lo} = x \& 0x0f \in \{0, \dots, 15\}, \quad x_{hi} = x \gg 4 \in \{0, \dots, 15\}. \quad (21)$$

Since $\bar{A}_{S_2}$ is a GF(2)-linear map it satisfies $\bar{A}_{S_2}(u \oplus v) = \bar{A}_{S_2}(u) \oplus \bar{A}_{S_2}(v)$. Applying this to (21):

$$\bar{A}_{S_2}(x) = \bar{A}_{S_2}(x_{lo} \oplus (x_{hi} \ll 4)) = \bar{A}_{S_2}(x_{lo}) \oplus \bar{A}_{S_2}(x_{hi} \ll 4). \quad (22)$$

Define the two 16-entry tables:

$$T^{lo}[j] := \bar{A}_{S_2}(j) \oplus c', \quad j \in \{0, \dots, 15\}, \quad (23)$$

$$T^{hi}[i] := \bar{A}_{S_2}(i \cdot 16), \quad i \in \{0, \dots, 15\}, \quad (24)$$

where c' is the constant from (18), absorbed into every entry of T^{lo} (since T^{hi} contains no constant offset). Then for every $x \in \{0, \dots, 255\}$,

$$\bar{A}_{S_2}(x) \oplus c' = T^{lo}[x_{lo}] \oplus T^{hi}[x_{hi}]. \quad (25)$$

Evaluated on the AESE output $x = S_1(\cdot)$, the right-hand side realizes S_2 .

Let $\mathbf{M} \in \text{GF}(2)^{8 \times 8}$ denote the matrix of $\bar{A}_{S_2}$ in the LSB-first basis $x = (x_0, x_1, \dots, x_7)^\top$ (bit k of x is x_k). Partition the columns of $\mathbf{M}$ as $\mathbf{M} = [\mathbf{M}_L \mid \mathbf{M}_H]$, where $\mathbf{M}_L$ comprises columns 0-3 (multiplied by x_{lo}) and $\mathbf{M}_H$ comprises columns 4-7 (multiplied by x_{hi}):

$$\mathbf{M}x = \mathbf{M}_L x_{lo} \oplus \mathbf{M}_H x_{hi}. \quad (26)$$

The 16 entries of T^{hi} are exactly the GF(2)-linear combinations of columns 4-7 of $\mathbf{M}$; the 16 entries of T^{lo} are the analogous combinations of columns 0-3 with the constant c' added to each entry.

The same recipe handles S_2^{-1} as well: applying the construction with $(\bar{A}, c) = (\bar{A}_{S_2^{-1}}, c'')$ from Eq. (20) gives two analogous tables whose XOR computes the affine pre-transform $\bar{A}_{S_2^{-1}}(y) \oplus c''$ required by Eq. (19) before AESD.

Each 16-entry table (23)–(24) occupies 16 bytes, fitting in a single 128-bit NEON register. We precompute four tables ($T_{S_2}^{lo}$, $T_{S_2}^{hi}$, $T_{S_2^{-1}}^{lo}$, $T_{S_2^{-1}}^{hi}$) and load them once into registers v_{19} – v_{22} ; they remain resident throughout all rounds.

The implementation for each register v_j : (1) AESE or AESD to obtain the composite-field inverse; (2) extract low nibble via AND with the nibble mask in v_{18} ; (3) extract high nibble via USHR #4; (4) look up low nibble with TBL; (5) look up high nibble with TBL; (6) combine

Table 2: S-box instruction cost per byte position (16 blocks).

S-box	Method	Instructions
S_1	AESE + SR^2 TBL	2
S_1^{-1}	AESD (no correction needed)	1
S_2	AESE + nibble decomposition + SR^2 TBL	7
S_2^{-1}	nibble decomposition + AESD	6

with EOR. This requires 6 instructions for S_2^{-1} and 7 for S_2 (the extra instruction being the SR^2 TBL correction after AESE), with zero memory accesses. Figure 5 illustrates the dataflow; the concrete AArch64 assembly for the nibble-split macro and the complete substitution-layer scheduling are given in Appendix B.

Table 2 summarizes the per-S-box instruction counts. Each round applies four S-box types to 4 byte positions each (16 total). The total per-round substitution cost is $4 \times 2 + 4 \times 1 + 4 \times 7 + 4 \times 6 = 64$ instructions, of which 8 are SR^2 TBL corrections that undo the SR side-effect of AESE.

A single SL macro serves both SL1 and SL2: the caller permutes the register groups so that $S_1 \leftrightarrow S_1^{-1}$ and $S_2 \leftrightarrow S_2^{-1}$ (Appendix B, Listing 3).

x86 (Yoo et al.) vs. ARMv8 (this work). On the x86 architecture, Intel’s GFNI extension provides the `vgf2p8affineqb` instruction, which evaluates any 8×8 GF(2)-linear affine map in a single clock cycle. Yoo et al. exploit this: `VAESENCLAST` extracts S_1 with an SR^{-1} correction, then one `vgf2p8affineqb` applies $\bar{A}_{S_2}$, yielding S_2 in two instructions with no memory accesses [YKC23].

The ARMv8 architecture provides no equivalent of GFNI. Evaluating the 8×8 GF(2)-linear map $\bar{A}_{S_2}$ without such an instruction requires a nibble-decomposition of the matrix into four 4×8 sub-maps, implemented via TBL4: four 64-byte constant tables loaded into four registers (8 `ld1` instructions per S-box call). Reloading these tables every round is the bottleneck underlying Eum et al.’s per-batch load count (Section 2) and the resulting $3.94\times$ slowdown relative to our work (Section 5.1). Our nibble-split approach realizes the same combined affine map $\bar{A}_{S_2}$ using only two permanent 16-byte NEON registers (one per nibble table), eliminating the TBL4 lookup entirely so that all four S-box evaluations execute without a memory load.

4.3 Diffusion Layer and Register Optimization

The ARIA diffusion layer applies the 16×16 binary involution matrix M to the 16-byte state. Each output y_i is the XOR of exactly 7 input bytes; a naïve implementation requires $16 \times 6 = 96$ EOR instructions. Using the four shared t -values defined in Equation (5), together with 8 pairwise sub-expressions $s_{jk} = x_j \oplus x_k$ shared across outputs, each output reduces to $y_i = t_a \oplus s_{jk} \oplus x_l$ (2 XORs). The resulting DIFFUSE macro consists of 52 two-operand EORs (12 for t -values, 8 for sub-expressions, 32 for outputs) plus 4 MOVs that save original values before in-place overwrites.

On processors with FEAT_SHA3 (including the Apple M1), the three-operand EOR3 instruction computes $V_d = V_n \oplus V_m \oplus V_a$ in a single cycle. Each t -value computation then costs 2 instructions instead of 3, and each output collapses to a single EOR3. This reduces the DIFFUSE instruction count from 56 (two-operand EOR only) to 36 (see Table 4), approximately a 36% reduction. On platforms without FEAT_SHA3 (e.g., the Cortex-A76), each EOR3 is replaced by two EORs, adding 20 instructions per round; the S-box and transpose layers are unaffected.

The in-place XOR tree consumes all 16 upper registers ($v_{16}–v_{31}$) as scratch: 12 for shared intermediate sub-expressions and 4 for saving original values before overwrites. The

Table 3: NEON register allocation across the three round-function phases.

Registers	Contents	Subst	Transpose	Diffusion
v_0 – v_{15}	cipher state (16 blocks)	S-box in/out	reorder	XOR in/out
v_{16}	all-zeros (AESE/AESD round key)			
v_{17}	SR^2 permutation table			
v_{18}	nibble mask (0x0f)			
v_{19}	S_2 low-nibble table	constants	preserved	scratch
v_{20}	S_2 high-nibble table			
v_{21}	S_2^{-1} low-nibble table			
v_{22}	S_2^{-1} high-nibble table			
v_{23}	free	—	free	scratch
v_{24} – v_{29}	—	—	scratch	scratch
v_{30} – v_{31}	—	—	—	scratch

Table 4: Instruction count per round (ARIA-128, 16-block batch).

Component	Instructions	Key operations
KEYXOR	22	$4 \times LD1 + 16 \times EOR + 2 \text{ addr}$
SUBLAYER	64	$8 \times AESE + 8 \times AESD + 8 \times NIBBLE_SBOX + 8 \times TBL (SR^2)$
DIFFUSE	36	$32 \times EOR/EOR3 + 4 \times MOV$
RELOAD	2	$2 \times LD1$ (S-box constants)
Total	124	

4 saves arise because M is an involution—every v_i is simultaneously input and output—and four circular dependencies ($v_8 \leftrightarrow v_{15}$, $v_9 \leftrightarrow v_{14}$, $v_{10} \leftrightarrow v_{13}$, $v_{11} \leftrightarrow v_{12}$) make it impossible to overwrite one without first saving the other. Listing 4 (Appendix B) shows the complete XOR tree. Table 3 summarizes the register usage across the three round-function phases.

The S-box constants are reloaded after each DIFFUSE call using two `ld1` instructions (7 registers, 112 bytes).

Table 4 summarizes the instruction count per round. For ARIA-128, rounds 1–11 each cost 124 instructions. Round 12 (final) omits DIFFUSE and RELOAD and adds a final KEYXOR, totaling approximately 108 instructions. Including the two transposes (64 `TRN` each) and I/O, the complete function is approximately 1620 instructions for 256 bytes.

5 Performance Evaluation and Comparison

5.1 Experimental Setup and Results

We evaluate our implementation on two ARMv8 platforms: (1) an Apple Mac Mini (2020) with the Apple M1 system-on-chip (Firestorm performance core, 3.2 GHz) running macOS, compiled with `clang -O3 -march=armv8.2-a+crypto+sha3`; and (2) a Raspberry Pi 5 with the Broadcom BCM2712 (ARM Cortex-A76, 2.4 GHz, 4 cores) running Linux, compiled with `gcc -O3 -march=armv8.2-a+crypto`. The Cortex-A76 implements ARMv8.2-A with Cryptographic Extensions but lacks `FEAT_SHA3`; all `EOR3` instructions are replaced by two-operand `EOR` pairs (see Section 4.3 for the +20 instruction/round impact). All measurements are for ARIA-128 (12 rounds), encrypting 16 blocks ($16 \times 16 = 256$ bytes) per invocation. We employ two independent measurement methodologies: (1) **PMU counters**—we read the ARM PMU cycle counter (`PMCCNTR_ELO`) via the `kperf` interface, averaging over 8 repetitions of 100,000 iterations; (2) **wall-clock timing**—100,000 warmup iterations

followed by 10,000,000 timed iterations, multiplying elapsed time by the nominal CPU frequency. Both methods agree within 0.8–1.0%, confirming no systematic bias.

Table 5 presents the results. Our implementation achieves **1.483 cpb (2.158 GB/s)** on a single core; decryption is identical due to the involution property.

Table 6 presents the Cortex-A76 results. Our implementation achieves **3.586 cpb** on the Cortex-A76, a $5.82\times$ speedup over the optimized reference C (Ref C2). The higher cpb compared to the M1 reflects both the absence of **EOR3** and the Cortex-A76’s narrower microarchitecture (4-wide decode vs. the M1’s 8-wide decode, smaller reorder buffer).

The Apple M1 Firestorm core’s approximately 630-entry reorder buffer [Fru20] means that only changes reducing the total dynamic instruction count (e.g., **EOR3** fusion, dead-constant elimination) produce measurable throughput improvements; structural reorganizations that preserve instruction count show no measurable difference, as the deep out-of-order engine absorbs scheduling variations.

Correction of Prior Results. Eum et al. [EKK⁺22] reported 0.573 cpb for their 16-block parallel ARIA implementation on the Apple M1. Through communication with the original authors and independent re-evaluation on identical hardware, we confirmed that the published figure reflects a unit-conversion discrepancy in the benchmark routine; the actual performance is 5.845 cpb (PMU) and 5.850 cpb (wall-clock), agreeing within 0.1%. We therefore adopt the independently measured 5.845 cpb as the baseline for comparison throughout this paper.

5.2 Multi-Thread Scalability

To evaluate practical throughput in a streaming encryption scenario, we measure CTR-mode performance with multiple threads on both platforms: the Apple M1 (4 Firestorm performance cores + 4 Icestorm efficiency cores, up to 8 threads) and the Cortex-A76 (4 cores, up to 4 threads). Each thread encrypts an independent CTR stream for 2.0 s; we report aggregate throughput in GB/s. As a baseline, we also include the KISA reference C implementations (Ref C1: 8-bit LUT scalar; Ref C2: 32-bit optimized scalar).

Table 7 presents the multi-thread scaling results.

On the M1, our implementation scales near-linearly from 1 to 4 threads ($1.76 \rightarrow 6.67$ GB/s, $3.79\times$), closely tracking the ideal $4\times$ scaling on the four Firestorm performance cores. Since the M1 has exactly four Firestorm cores, the 4-thread figure of **6.67 GB/s** reflects the full performance-cluster throughput; at this point the gap over reference C widens to $8.78\times$ vs. Ref C2 (6.67 vs. 0.76 GB/s) and $10.76\times$ vs. Ref C1 (6.67 vs. 0.62 GB/s). Beyond 4 threads, the additional Icestorm efficiency cores contribute diminishing returns due to their narrower microarchitecture, yielding **8.33 GB/s** at 8 threads ($4.73\times$ vs. single-thread).

On the Cortex-A76, the implementation reaches **2.36 GB/s** with all 4 cores ($3.87\times$ vs. single-thread), showing near-linear scaling despite the absence of **EOR3** and a lower clock

Table 5: Performance of ARIA-128 on Apple M1 (single-thread, 3.2 GHz). [†]Published figure; our re-evaluation yields 5.845 cpb (Section 5.1).

Implementation	Technique	PMU (cpb)	Wall (cpb)	GB/s
KISA Ref C1 [Kor04]	Scalar, 8-bit LUT	18.657	18.696	0.172
KISA Ref C2 [Kor04]	Scalar, 32-bit opt. C	15.039	15.039	0.213
Eum et al. [EKK ⁺ 22] pub. [†]	S-box TBL + LD4/ST4	0.573	—	5.58
Eum et al. [EKK ⁺ 22] 16PT	S-box TBL + LD4/ST4	5.845	5.850	0.548
Ours	AESE/AESD + nibble TBL	1.483	1.484	2.158

Table 6: Performance of ARIA-128 on Cortex-A76 (Raspberry Pi 5, 2.4 GHz).

Implementation	PMU (cpb)	Wall (cpb)	GB/s
KISA Ref C1 [Kor04]	74.027	74.105	0.032
KISA Ref C2 [Kor04]	20.834	20.861	0.115
Eum 16PT [EKK ⁺ 22]	14.160	14.187	0.169
Ours	3.586	3.590	0.669

frequency (2.4 GHz vs. 3.2 GHz). At 4 threads, the speedup over Ref C2 is $5.24\times$ (2.36 vs. 0.45 GB/s) and over Ref C1 is $18.15\times$ (2.36 vs. 0.13 GB/s).

6 Conclusion

We presented an efficient ARIA implementation for ARMv8 that exploits the AESE/AESD cryptographic extensions to evaluate all four S-boxes. The key technical insight is that all four ARIA S-boxes reduce to a single computational primitive—the AES-field multiplicative inverse—which AESE/AESD compute in hardware. For S_1/S_1^{-1} , this mapping is direct (Eqs. (12)–(13)); for S_2/S_2^{-1} , we derived combined affine transformations $\bar{A}_{S_2}$ and $\bar{A}_{S_2^{-1}}$ via the Frobenius automorphism $\sigma_3 : y \mapsto y^8$ (and its inverse σ_5) of the AES field, applying the affine *after* AESE for S_2 and *before* AESD for S_2^{-1} , and realized each through a nibble-split decomposition (7 instructions for S_2 , 6 for S_2^{-1}) that fits entirely in two NEON registers. This eliminates the 192 S-box table loads per-encryption required by the prior TBL4 approach [EKK⁺22], yielding a **$3.94\times$ speedup (1.483 vs. 5.845 cpb on the Apple M1)**. The SR side-effect of AESE/AESD is reduced to a single SR^2 TBL per AESE-based S-box, exploiting $\text{SR}^4 = I$ to align the AESE and AESD groups in a common lane state that the key schedule absorbs.

On the Cortex-A76 (Raspberry Pi 5), the same implementation achieves **3.586 cpb** ($5.82\times$ over reference C) despite the absence of EOR3, confirming that the approach is portable across ARMv8 microarchitectures with Cryptographic Extensions. Multi-threaded CTR-mode measurements show near-linear scaling on both platforms: **6.67 GB/s** at 4 threads on the M1 and **2.36 GB/s** at 4 threads on the Cortex-A76.

Code Availability. The complete AArch64 assembly implementation, benchmark harness, and correctness test suite are available at https://github.com/dampers/ARIA_on_armv8.

Table 7: Multi-thread CTR-mode throughput (GB/s, T: Threads).

Platform	Implementation	1T	2T	4T	8T
Apple M1 (3.2 GHz, 4+4 cores)	KISA Ref C1 [Kor04]	0.16	0.32	0.62	0.83
	KISA Ref C2 [Kor04]	0.20	0.40	0.76	1.14
	Eum 16PT [EKK ⁺ 22]	0.52	1.02	1.96	2.50
	Ours	1.76	3.46	6.67	8.33
Cortex-A76 (2.4 GHz, 4 cores)	KISA Ref C1 [Kor04]	0.03	0.06	0.13	—
	KISA Ref C2 [Kor04]	0.11	0.23	0.45	—
	Eum 16PT [EKK ⁺ 22]	0.17	0.33	0.65	—
	Ours	0.61	1.18	2.36	—

Acknowledgements. This research was supported in part by Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education(No. 25411243) and in part by 2026 Research Grant from Kangwon National University.

We would like to thank Eum et al. for providing their source code and for the constructive discussions regarding their prior work.

References

- [Arm24] Arm Ltd. Arm architecture reference manual for A-profile architecture, 2024. URL: <https://developer.arm.com/documentation/ddi0487/latest/>.
- [BS12] Daniel J. Bernstein and Peter Schwabe. NEON crypto. In *Cryptographic Hardware and Embedded Systems – CHES 2012*, volume 7428 of *LNCS*, pages 320–339. Springer, 2012. doi:10.1007/978-3-642-33027-8_19.
- [EKK⁺22] Siwoo Eum, Hyunjun Kim, Hyeokdong Kwon, Minjoo Sim, Gyeongju Song, and Hwajeong Seo. Parallel implementations of ARIA on ARM processors and graphics processing unit. *Applied Sciences*, 12(23):12246, 2022. doi:10.3390/app122312246.
- [ESKS25] Siwoo Eum, Minhong Song, Sangwon Kim, and Hwajeong Seo. Efficient GPU parallel implementation and optimization of ARIA for counter and exhaustive key-search modes. *Electronics*, 14(10):2021, 2025. doi:10.3390/electronics14102021.
- [ESSS25] Siwoo Eum, Minhong Song, Minjoo Sim, and Hwajeong Seo. Maximizing ARIA-CTR performance through kernel-level memory optimization on Apple M1 GPU. In *Information Security and Cryptology – ICISC 2025*, 2025.
- [Fru20] Andrei Frumusanu. The 2020 mac mini unleashed: Putting Apple silicon M1 to the test. AnandTech, 2020. URL: <https://www.anandtech.com/show/16252/mac-mini-apple-m1-tested>.
- [GL15] Conrado P. L. Gouvêa and Julio López. Implementing GCM on ARMv8. In *Topics in Cryptology – CT-RSA 2015*, volume 9048 of *LNCS*, pages 167–180. Springer, 2015. doi:10.1007/978-3-319-16715-2_9.
- [Guo22] Weiji Guo. Efficient constant-time implementation of SM4 with Intel GFNI instruction set extension and Arm NEON coprocessor. Cryptology ePrint Archive, Paper 2022/1154, 2022. URL: <https://eprint.iacr.org/2022/1154>.
- [Joh21] Dougall Johnson. Apple M1 microarchitecture research, 2021. URL: <https://dougallj.github.io/applecpu/firestorm.html>.
- [KKE⁺22] Hyeokdong Kwon, Hyunjun Kim, Siwoo Eum, Minjoo Sim, Hyunji Kim, Wai-Kong Lee, Zhi Hu, and Hwajeong Seo. Optimized implementation of SM4 on AVR microcontrollers, RISC-V processors, and ARM processors. *IEEE Access*, 10:80225–80233, 2022. doi:10.1109/ACCESS.2022.3195217.
- [KKP⁺04] Daesung Kwon, Jaesung Kim, Sangwoo Park, Soo Hak Sung, Yaekwon Sohn, Jung Hwan Song, Yongjin Yeom, E-Joong Yoon, Sangjin Lee, Jaewon Lee, Seongtaek Chee, Daewan Han, and Jin Hong. New block cipher: ARIA. In *Information Security and Cryptology – ICISC 2003*, volume 2971 of *LNCS*, pages 432–445. Springer, 2004. doi:10.1007/978-3-540-24691-6_32.

- [KKS21] Youngjoo Kwak, Yujin Kim, and Seog Chung Seo. Benchmarking korean block ciphers on 32-bit RISC-V processor. *Journal of the Korea Institute of Information Security and Cryptology*, 31(3):331–340, 2021.
- [KLPK11] Woo-Hwan Kim, Jungkeun Lee, Je-Hong Park, and Daesung Kwon. Addition of the ARIA cipher suites to Transport Layer Security (TLS). RFC 6209, 2011. [doi:10.17487/RFC6209](https://doi.org/10.17487/RFC6209).
- [Kor04] Korea Internet and Security Agency. ARIA algorithm specification, 2004. KISA-KS-X-1213-1.
- [KS09] Emilia Käsper and Peter Schwabe. Faster and timing-attack resistant AES-GCM. In *Cryptographic Hardware and Embedded Systems – CHES 2009*, volume 5747 of *LNCS*, pages 1–17. Springer, 2009. [doi:10.1007/978-3-642-04138-9_1](https://doi.org/10.1007/978-3-642-04138-9_1).
- [Lee10] Jungkeun Lee. A description of the ARIA encryption algorithm. RFC 5794, 2010. [doi:10.17487/RFC5794](https://doi.org/10.17487/RFC5794).
- [LSS⁺25] Myeonghoon Lee, Hanbeom Shin, Myongsu Shin, Dongjae Lee, Seog Chung Seo, and Seokhie Hong. Best interleave size of GIFT-128 on 64-bit ARM processor. *Journal of Cryptographic Engineering*, 15(1):6, 2025. [doi:10.1007/s13389-025-00372-w](https://doi.org/10.1007/s13389-025-00372-w).
- [Nat01] National Institute of Standards and Technology. Advanced encryption standard (AES), 2001. [doi:10.6028/NIST.FIPS.197](https://doi.org/10.6028/NIST.FIPS.197).
- [SKJ⁺21] Hwajeong Seo, Hyunjun Kim, Kyungbae Jang, Hyeokdong Kwon, Minjoo Sim, Gyeongju Song, and Sungkyu Uhm. Compact implementation of ARIA on 16-bit MSP430 and 32-bit ARM Cortex-M3 microcontrollers. *Electronics*, 10(8):908, 2021. [doi:10.3390/electronics10080908](https://doi.org/10.3390/electronics10080908).
- [SS20] Jingyo Song and Seog Chung Seo. Secure and fast implementation of ARX-based block ciphers using ASIMD instructions in ARMv8 platforms. *IEEE Access*, 8:193138–193153, 2020. [doi:10.1109/ACCESS.2020.3032785](https://doi.org/10.1109/ACCESS.2020.3032785).
- [YIK24] Dongwook Yun, Muhammad Iqbal, and Howon Kim. Optimization of ARIA block cipher implementation on RISC-V systems through instruction extension. In *2024 International Conference on Platform Technology and Service (PlatCon)*, pages 197–200. IEEE, 2024. [doi:10.1109/PlatCon63925.2024.10830667](https://doi.org/10.1109/PlatCon63925.2024.10830667).
- [YKC23] Tae-Hee Yoo, Jussi Kivilinna, and Choong-Hee Cho. AVX-based acceleration of ARIA block cipher algorithm. *IEEE Access*, 11:77403–77415, 2023. [doi:10.1109/ACCESS.2023.3298026](https://doi.org/10.1109/ACCESS.2023.3298026).

A Explicit Form of the Combined Affine Maps

This appendix gives concrete numerical values for the matrices that appear in the combined-affine derivation of Section 4.2 (Equations (18) and (20)), so that no derivation from the ARIA [KKP⁺04] or AES [Nat01] specifications is required.

We adopt the LSB-first convention: an 8×8 binary matrix A acts on a byte $x = \sum_{k=0}^7 x_k 2^k$ via $(Ax)_i = \bigoplus_{j=0}^7 A_{ij} x_j$, with column j (row i) indexing input (output) bit j (i). With this convention, the AES inverse affine matrix M_{AES}^{-1} [Nat01], the ARIA matrix B [KKP⁺04], and the Frobenius matrix C representing $\sigma_3 : y \mapsto y^8$ on the standard basis $\{1, x, \dots, x^7\}$ of $\text{GF}(2^8)/p_1$ (column j of C is $x^{8j} \bmod p_1$) are

$$\begin{aligned} M_{\text{AES}}^{-1} &= \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}, \\ B &= \begin{pmatrix} 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 1 \end{pmatrix}, \\ C &= \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{pmatrix}. \end{aligned} \tag{27}$$

Multiplying these three matrices over $\text{GF}(2)$ yields the combined affine of Eq. (18):

$$\begin{aligned} \bar{A}_{S_2} &= B \cdot C \cdot M_{\text{AES}}^{-1} = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}, \\ c' &= \bar{A}_{S_2} \cdot c_{\text{AES}} \oplus b = \text{0x88}. \end{aligned} \tag{28}$$

For the inverse direction, inverting B , C , and M_{AES}^{-1} (which yields M_{AES}) and applying Eq. (20):

$$\begin{aligned} \bar{A}_{S_2^{-1}} &= M_{\text{AES}} \cdot C^{-1} \cdot B^{-1} = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}, \\ c'' &= \bar{A}_{S_2^{-1}} \cdot b \oplus c_{\text{AES}} = \text{0x04}. \end{aligned} \tag{29}$$

Exhaustive enumeration over all 256 inputs confirms both identities (Section 4.2); the 16-entry tables $T_{S_2}^{\text{lo}}, T_{S_2}^{\text{hi}}, T_{S_2^{-1}}^{\text{lo}}, T_{S_2^{-1}}^{\text{hi}}$ used by the nibble-split implementation (Equations (23)–(24)) are obtained by evaluating $\bar{A}_{S_2}(\cdot) \oplus c', \bar{A}_{S_2}(\cdot), \bar{A}_{S_2^{-1}}(\cdot) \oplus c'', \bar{A}_{S_2^{-1}}(\cdot)$ on the 16 low-nibble inputs $\{0, \dots, 15\}$ and on the 16 high-nibble inputs $\{0, 16, 32, \dots, 240\}$, respectively.

B Implementation Listings

This appendix collects the AArch64 NEON assembly for the three round-function phases (transpose, S-box, diffusion) described in Sections 4.1–4.3.

Listing 1: TRANSPOSE16 fused-stages butterfly (Section 4.1).

```
// Input: v0=[b_{0,0..15}], v1=[b_{1,0..15}], ...

// == Pass A (group 0): .16b then .8h ==
// Before: v0 = b_{0,*}, v1 = b_{1,*}
trn1 t0.16b, v0.16b, v1.16b // t0 = [b0,0 b1,0 b0,2 b1,2 ...]
trn2 t1.16b, v0.16b, v1.16b // t1 = [b0,1 b1,1 b0,3 b1,3 ...]
trn1 t2.16b, v2.16b, v3.16b // t2, t3: same for v2,v3
trn2 t3.16b, v2.16b, v3.16b
trn1 v0.8h, t0.8h, t2.8h // v0 = [b0,0 b1,0 b2,0 b3,0 ...]
trn2 v2.8h, t0.8h, t2.8h // v2 = [b0,2 b1,2 b2,2 b3,2 ...]
trn1 v1.8h, t1.8h, t3.8h // v1 = [b0,1 b1,1 b2,1 b3,1 ...]
trn2 v3.8h, t1.8h, t3.8h // v3 = [b0,3 b1,3 b2,3 b3,3 ...]
// ... groups 1-3: same pattern on {v4-v7}, {v8-v11}, {v12-v15}
// After Pass A: 4x4 sub-blocks are transposed

// == Pass B (column 0): .4s then .2d ==
// Before: v0 has blocks 0-3, v4 has blocks 4-7, etc.
trn1 t0.4s, v0.4s, v4.4s // merge blocks 0-3 with 4-7
trn2 t4.4s, v0.4s, v4.4s
trn1 t1.4s, v8.4s, v12.4s // merge blocks 8-11 with 12-15
trn2 t5.4s, v8.4s, v12.4s
trn1 v0.2d, t0.2d, t1.2d // v0 = byte 0 from all 16 blocks
trn2 v8.2d, t0.2d, t1.2d // v8 = byte 8 from all 16 blocks
trn1 v4.2d, t4.2d, t5.2d // v4 = byte 4 from all 16 blocks
trn2 v12.2d, t4.2d, t5.2d // v12 = byte 12 from all 16 blocks
// ... columns 1-3: same pattern on {v1,v5,v9,v13}, etc.
```

Listing 2: NIBBLE_SBOX macro for S_2/S_2^{-1} (Eq. (25)).

```
and tmp0.16b, src.16b, nib_mask.16b // tmp0 = x_lo
ushr tmp1.16b, src.16b, #4 // tmp1 = x_hi
tbl tmp0.16b, {lo_tbl.16b}, tmp0.16b // tmp0 = T_lo[x_lo]
tbl tmp1.16b, {hi_tbl.16b}, tmp1.16b // tmp1 = T_hi[x_hi]
eor dst.16b, tmp0.16b, tmp1.16b // dst = S(x)
```

Listing 3: Substitution-layer instructions per round (64 total; the SL macro contains the first 56, with the trailing 8 TBL corrections deferred to the start of DIFFUSE for scheduling).

```
// == SL macro begins (56 instr) ==
// S1: AESE (4 instr)
aese s1a.16b, kzero.16b    ... // x4
// S2 step 1: AESE (4 instr)
aese s2a.16b, kzero.16b    ... // x4
// S1^{-1}: AESD (4 instr)
aesd s3a.16b, kzero.16b    ... // x4
// S2^{-1} step 1: nibble BEFORE AESD (4x5 instr)
NIBBLE_SBOX s4a, s4a, knib, ksb4lo, ksb4hi, t0, t1 ...
// S2^{-1} step 2: AESD (4 instr)
aesd s4a.16b, kzero.16b    ... // x4
// S2 step 2: nibble AFTER AESE (4x5 instr)
NIBBLE_SBOX s2a, s2a, knib, ksb2lo, ksb2hi, t0, t1 ...
// == SL macro ends; DIFFUSE prefix below (8 instr) ==
// SR^2 TBL on S1+S2 groups: SR^2 o SR = SR^{-1}
tbl s1a.16b, {s1a.16b}, ksr2.16b ... // x4
tbl s2a.16b, {s2a.16b}, ksr2.16b ... // x4
// Total: 4+4+4+20+4+20+8 = 64 instructions
```

Listing 4: DIFFUSE XOR-tree structure (EOR3 variant, Section 4.3).

```
// (1) Group-A: t-values via EOR+EOR3 (8 instr)
eor ta0.16b, b3.16b, b4.16b
eor3 ta0.16b, ta0.16b, b9.16b, b14.16b // t0
... // ta1=t1, ta2=t2, ta3=t3 analogous

// (2) Group-C: high-block XOR pairs (4 instr)
eor tc0.16b, b8.16b, b13.16b
... // tc1, tc2, tc3

// (3) Group-B: mixed XOR pairs (4 instr)
eor tb0.16b, b1.16b, b4.16b
... // tb1, tb2, tb3

// (4) Output group 0: y0,y6,y2,y4 (5 instr)
mov s0.16b, b0.16b // save before overwrite
eor3 b0.16b, tc0.16b, ta0.16b, b6.16b // y0=t0^x6^x8^x13
eor3 b6.16b, tc1.16b, b2.16b, ta3.16b // y6
eor3 b2.16b, tc2.16b, b4.16b, ta2.16b // y2
eor3 b4.16b, tc3.16b, ta1.16b, s0.16b // y4 (uses saved b0)
... // groups 1-3: same MOV+4xEOR3 pattern (15 instr)
```